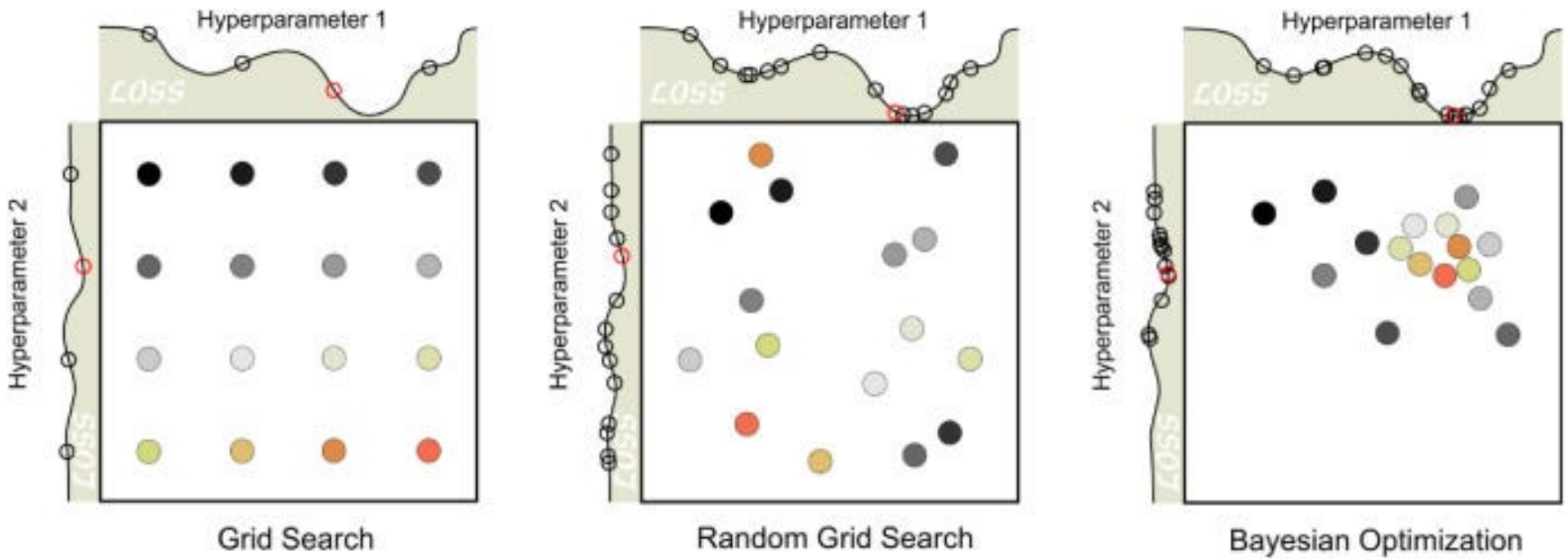
AutoML, Autoresearch, MLOps +@

26.4.7

서민교

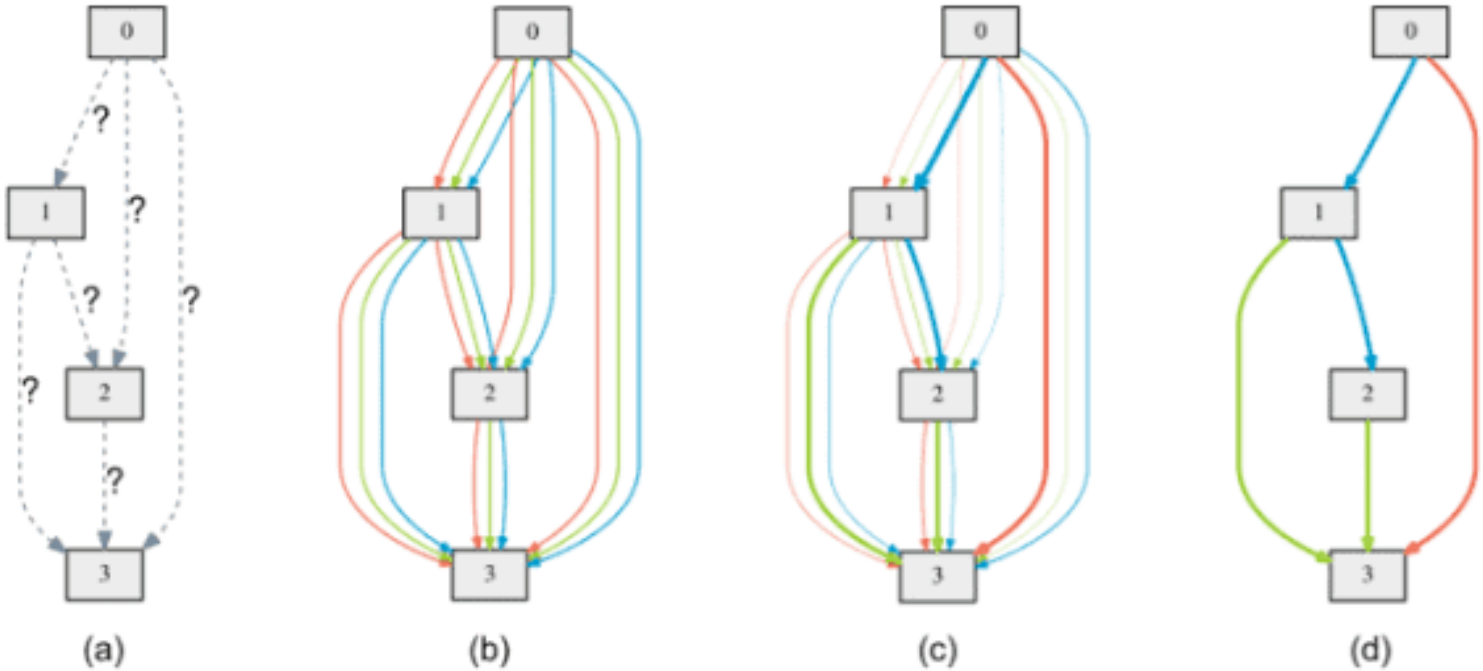
1. AutoML이란 무엇인가?

- 모델 개발 탐색 일부 자동화
- 대표 대상: `model selection`, `hyperparameter tuning`, `pipeline search`
- 핵심: 비교적 주어진 `search space` 안 최적 설정 탐색



2. `Neural Architecture Search`는 AutoML의 확장이다

- parameter 대신 architecture 탐색
- AutoML의 `더 넓은 search space` 확장선
- 그래도 중심은 여전히 모델/파이프라인 후보 탐색



`hyperparameter search → pipeline search → architecture search`

3. `Autoresearch`는 어떻게 등장했고 무엇이 다른가?

- karpathy/autoresearch: 작은 training setup 위 `read → edit → run → keep-or-revert` loop 제시
- 연구 workflow 일부를 agent가 직접 수행하며, 설정 탐색을 넘어 `code`, `module`, `experiment` 자체 수정
- AutoML의 `fixed search space` 바깥으로 확장
- 이후 RD-Agent, AI-Scientist, GPT Researcher 등으로 빠르게 확장

4. Agent 작업 흐름

- 코드 읽기, baseline 파악
- 작은 가설 하나 선택
- 학습 코드나 설정 수정
- 짧은 실험 실행, metric 확인
- 나쁘면 revert, 의미 있으면 keep
- 핵심: `edit 한 번`이 아니라 `짧은 실험 loop의 누적`

```
Question → Read → Edit → Run → Analyze → Next experiment
```

5. AutoML vs. Autoresearch

항목	AutoML	Autoresearch
탐색 대상	config, pipeline, architecture	hypothesis, code, module, experiment
핵심 질문	어떤 설정이 가장 좋은가	다음에 어떤 실험을 해야 하는가
edit 단위	parameter / architecture	code / module / pipeline / experiment
평가 방식	objective 중심	objective + reasoning + iteration
위험	비효율적 탐색	incoherent search, metric hacking
필요한 인프라	experiment infra	experiment + memory + harness

6. 사용례와 확장

사용례

- 문헌 조사 / deep research: GPT Researcher
- 코드 수정 + 실험 반복: karpathy/autoresearch, RD-Agent
- end-to-end 연구 자동화: AI-Scientist

확장

- benchmark / evaluation: MLE-bench, MLAgentBench, MLR-Bench
- plugin / skill 생태계: awesome-autoresearch, Awesome Auto Research Tools
- memory, reusable modules, hardware fork

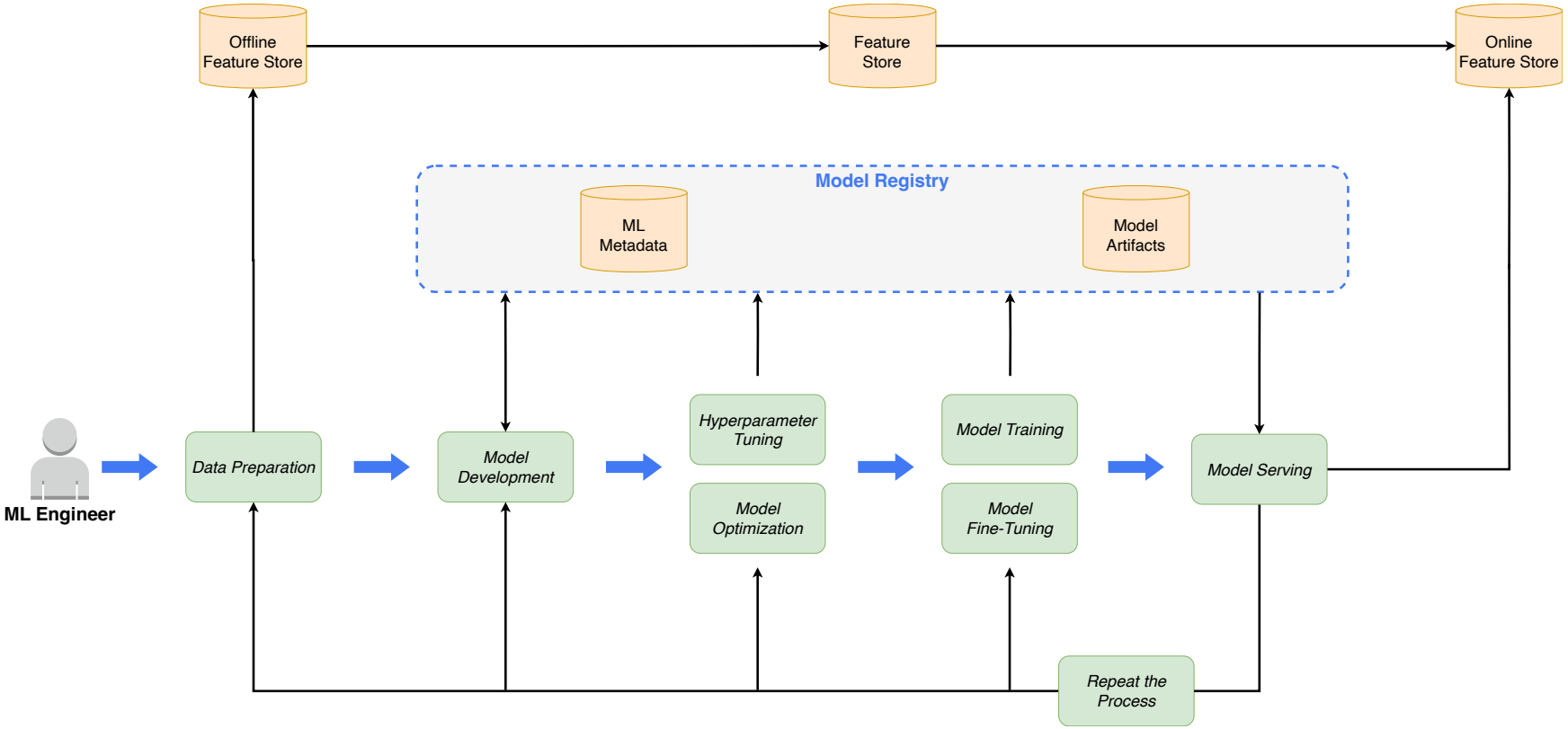
7. 체계적인 실험 관리의 필요

- 공통 문제: `많은 run` 비교와 누적
- 필수 요소: `tracking`, `lineage`, `orchestration`
- agent edit가 들어오면 `artifact`, `promotion`, `monitoring`, `cost control` 중요도 상승
- 결국 운영 문제

8. AutoML과 Autoresearch가 공통으로 요구하는 MLOps 요소

요소	AutoML에서의 역할	Autoresearch에서의 역할
tracking	sweep 비교	hypothesis / code edit history 비교
orchestration	search job 실행	agent + eval job 실행
registry / lineage	best model 승격	experiment / prompt / code provenance 보존
monitoring / cost	retrain trigger, SLO	budget, drift, unsafe promotion guardrail

9. MLOps는 모델 개발, 관리, 배포 파이프라인을 유지 관리하는 작업이다



- Autoresearch loop는 이 큰 ML lifecycle 안의 일부
- 실제 시스템: `data`, `experiment`, `model registry`, `deployment`, `monitoring`
- 핵심 역할: `지속 운영`, `추적`, `승격`, `유지관리`

10. Autoresearch의 단점

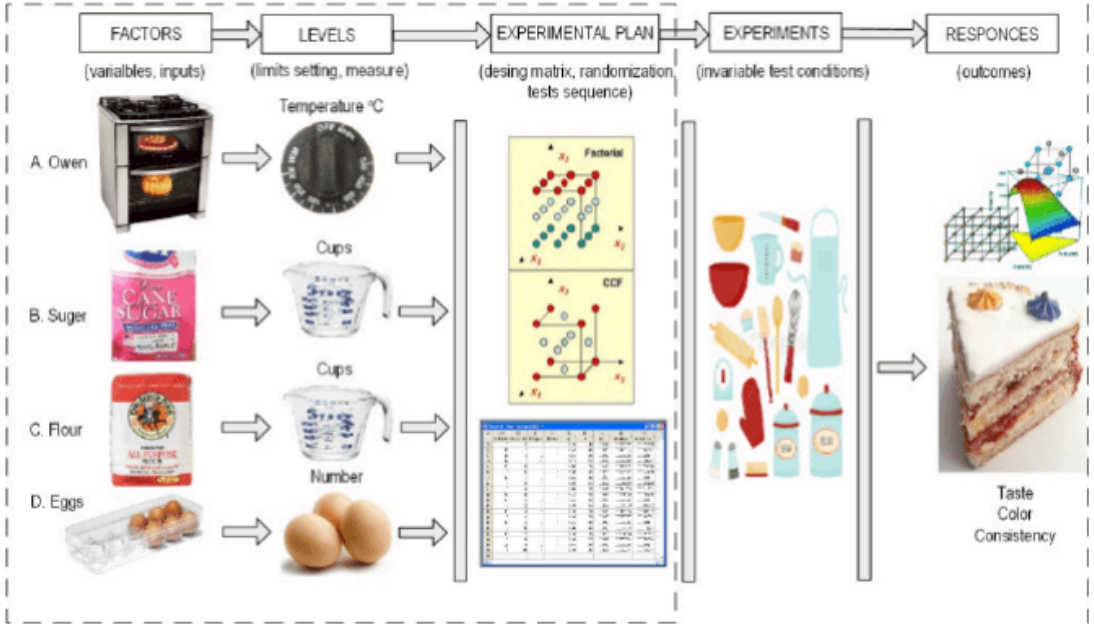
- 실험이 즉흥적으로 이어지기 쉬움
- 왜 이 실험을 했는지 attribution이 약함
- 큰 수정, 작은 튜닝, 검증 실험이 섞이기 쉬움
- robustness, replication, interaction 확인이 뒤로 밀림
- 잘 정리된 random search로 퇴화할 위험

11. 어떤 Harness가 필요한가

- 무엇을 먼저 볼지 정하는 우선순위
- 어떤 조합을 함께 볼지 정하는 규율
- 탐색 단계와 검증 단계 분리
- 작은 수정과 큰 수정을 다르게 다루는 운영 규칙
- 실패도 정보로 남기는 구조
- 다음 라운드를 설계하는 순차 실험 체계

12. Design of Experiments(DoE)란 무엇인가

- 여러 요인을 한 번에 바꿔 보며 effect를 읽는 실험 설계
- 한 번의 최고점보다 `요인`, `상호작용`, `안정성` 파악에 강점
- 핵심 질문: 무엇을 바꿨고, 무엇이 실제로 영향을 줬는가



13. DoE에서 빌려오는 개념

- `screening` : 중요한 요인부터 좁히기
- `factorial thinking` : interaction 보기
- `sequential design` : 라운드별 정교화
- `robust design` : 평균이 아니라 안정성까지 확인
- `mixture / allocation` : 예산과 비율 배분

14. DoE-guided 운영과 비교한 Agents

Agent	운영 방식	특징
`01 Sequential`	Vanilla autoresearch	최신 기록에 맞춰 작은 변경을 순차 적용
`02 Simple DoE`	DoE-guided screening	factor와 level을 두고 screening 중심 비교
`03 Advanced DoE Tic-Tac-To`	DoE-guided staged program	staged DoE + 실험 타입 예산 분배

- `Tic / Tac / To` : 다중 모듈 변경 / 단일 모듈 교체 / 같은 구조 안 수치 조정, 목표 비율 `1:2:4`

15. 실험 설정

항목	설정
benchmarks	`cifar10_real`, `twenty_newsgroups_real`
data budget	CIFAR-10 `max_samples=4000`, 20 Newsgroups `max_samples=8000`
model	`mlp`
agents	`01 Sequential`, `02 Simple DoE`, `03 Advanced DoE + Tic-Tac-To`
execution	agent별 isolated root에서 validation `200` rounds 후 hidden finalize

실행 조건

- agent별 isolated root를 따로 만들어 context leakage 없이 독립 실행
- 같은 start control에서 출발해 validation-only `200` runs 누적
- hidden test는 탐색 종료 뒤 `finalize-agent`에서만 공개
- text는 고정 TF-IDF 표현 위에서 MLP만 탐색하고, 이미지도 projection 없이 raw feature 위 MLP만 탐색

NN-only curated search surface: `8` knobs

- preprocessing: `normalization`
- model structure: `hidden_dims`
- optimization: `activation`, `solver`, `learning_rate_init`, `batch_size`

16. CIFAR-10 결과: validation 탐색과 hidden test

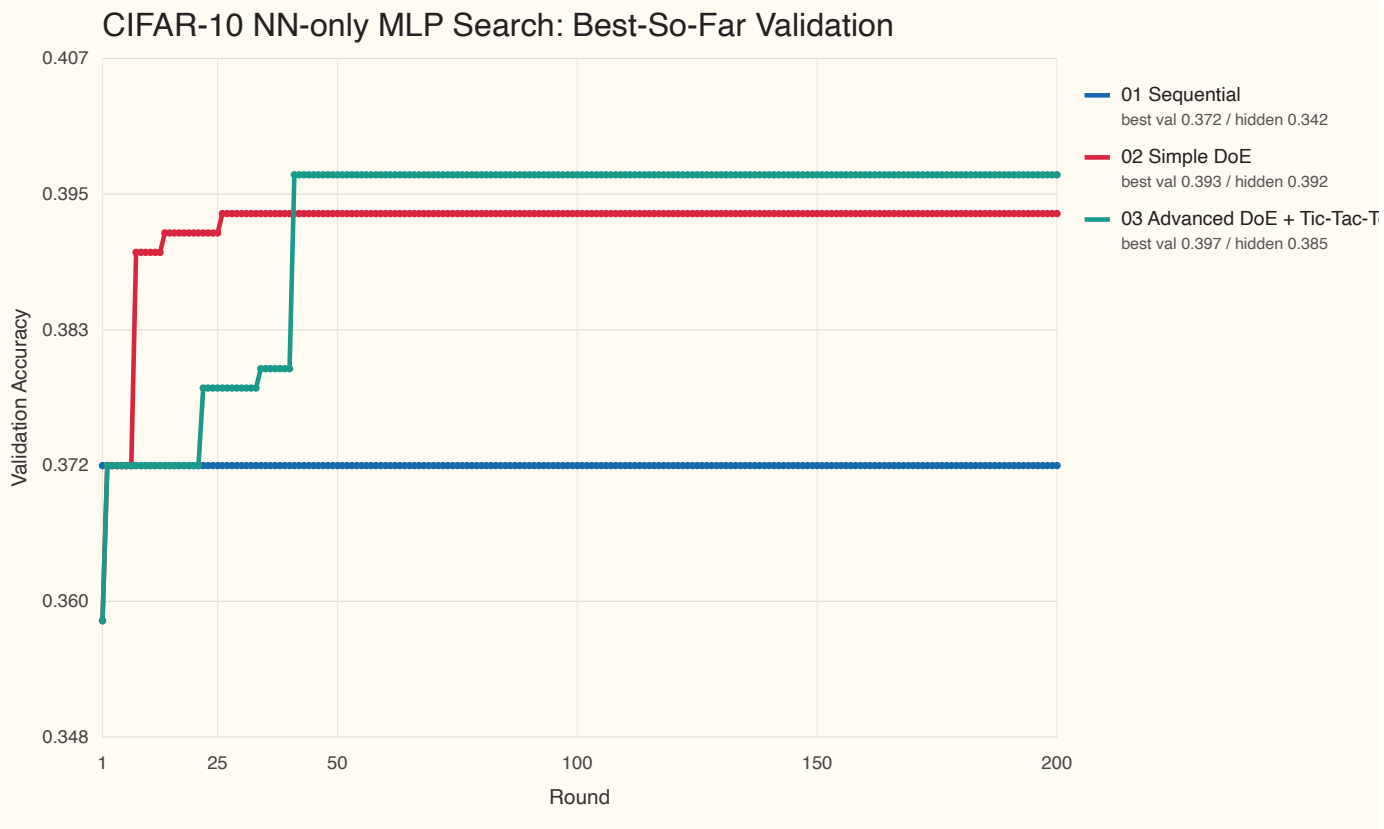
조건: `cifar10_real` / `mlp` / curated `8` knobs / validation `200` runs + hidden finalize

Agent	Best Val	Hidden Test	Gap	Run of Best	Incumbent Updates
`01 Sequential`	`0.3717`	`0.3417`	`0.0300`	`1`	`1`
`02 Simple DoE`	`0.3933`	`0.3917`	`0.0017`	`26`	`5`
`03 Advanced DoE + Tic-Tac-To`	`0.3967`	`0.3850`	`0.0117`	`41`	`5`

대표 config

- `01 Sequential`: `standard + [64,32] + relu + adam + no internal norm + wd=0.0005 + lr=0.001 + bs=64`
- `02 Simple DoE`: `standard + [64,64,64] + relu + adam + batchnorm + wd=0.001 + lr=0.001 + bs=32`
- `03 Advanced DoE + Tic-Tac-To`: `maxabs + [32,64] + leaky_relu + adam + layernorm + wd=0.0008 + lr=0.0012 + bs=128`

17. CIFAR-10 결과: 탐색 궤적



- `Sequential` : `run 1` 이후 거의 움직이지 못했다.
- `Simple DoE` : `run 26` 에 best를 찾았고 hidden 유지력이 가장 좋았다.
- `Advanced DoE + Tic-Tac-To` : `run 41` 까지 개선했고, `Tic:Tac:To = 29:57:113` 으로 목표 `1:2:4` 에 근접했다.

18. 20 Newsgroups 결과: validation 탐색과 hidden test

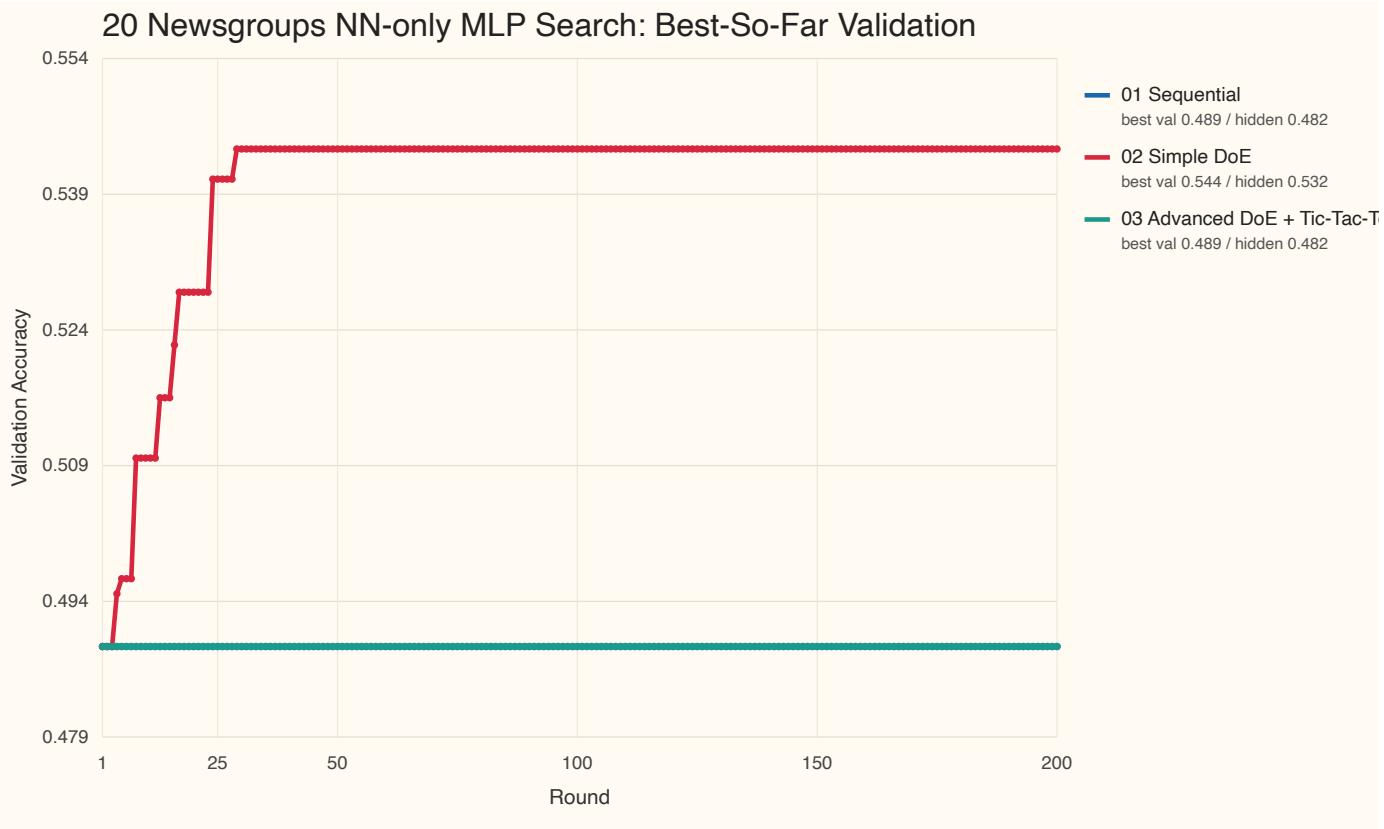
조건: `twenty_newsgroups_real` / `mlp` / curated `8` knobs / validation `200` runs + hidden finalize

Agent	Best Val	Hidden Test	Gap	Run of Best	Incumbent Updates
`01 Sequential`	`0.4892`	`0.4825`	`0.0067`	`1`	`1`
`02 Simple DoE`	`0.5442`	`0.5325`	`0.0117`	`29`	`9`
`03 Advanced DoE + Tic-Tac-To`	`0.4892`	`0.4825`	`0.0067`	`1`	`1`

대표 config

- `01 Sequential`: `maxabs + [64,32] + relu + adam + layernorm + wd=0.0005 + lr=0.001 + bs=64`
- `02 Simple DoE`: `standard + [64] + tanh + adam + layernorm + wd=0.0005 + lr=0.002 + bs=64`
- `03 Advanced DoE + Tic-Tac-To`: `maxabs + [64,32] + relu + adam + layernorm + wd=0.0005 + lr=0.001 + bs=64`

19. 20 Newsgroups 결과: 탐색 궤적



- `Sequential`과 `Advanced`는 baseline 근처에 머물렀다.
- `Simple DoE`만 early screening으로 basin을 바꿨고 그 우위를 유지했다.
- 이 text/TF-IDF 공간에서는 interaction 탐색보다 first-order screening payoff가 더 컸다.

20. 두 dataset을 같이 보면

- CIFAR: highest validation은 `Advanced`, hidden best는 `Simple DoE`
- Text: validation과 hidden 모두 `Simple DoE` 우세
- 해석: image-like space에선 staged search가, text-like space에선 screening이 더 잘 맞았다.
- `Sequential`은 두 dataset 모두 incumbent trap에 취약했다.

21. 히스토리와 피드백에서 추출한 지식

`01 Sequential`

- 빠른 sanity check와 baseline 확보에는 유용하다.
- 첫 incumbent가 local optimum이면 장기 예산을 줘도 잘 못 빠져나온다.

`02 Simple DoE`

- CIFAR prior: `standard + [64,64,64] + relu + adam + batchnorm + wd=0.001 + lr=0.001 + bs=32`
- Text prior: `standard + [64] + tanh + adam + layernorm + wd=0.0005 + lr=0.002 + bs=64`
- 두 dataset 모두 hidden transfer가 가장 안정적이었다.

`03 Advanced DoE + Tic-Tac-To`

- CIFAR prior: `maxabs + [32,64] + leaky_relu + adam + layernorm + wd=0.0008 + lr=0.0012 + bs=128`
- text에서는 staged budget이 basin 전환으로 이어지지 못했다.
- `Tic/Tac/To` 예산은 image-like search space에서 더 유효했다.

22. 히스토리 진단: baseline보다 loop quality 문제가 더 컸다

Agent	CIFAR trace	Text trace	진단
`Sequential`	best가 `run 1`	best가 `run 1`	incumbent replay, 탐색 붕괴
`Simple DoE`	`run 26` 까지 개선	`run 29` 까지 개선	screening은 의미 있었고 후반은 다소 중복
`Advanced DoE + Tic-Tac-To`	`run 41` 까지 개선	best가 `run 1`	CIFAR에선 유효, text에선 backend-safe anchor에 고착

- `baseline이 너무 강해서 못 넘었다` 는 설명은 충분하지 않다.
- 실제로는 일부 agent가 중간부터 같은 안전 설정을 반복 제출하며 탐색이 멈췄다.
- 다음 harness 개선 우선순위는 `더 똑똑한 전략` 보다 `중복 submission 방지`, `backend-fix loop 차단`, `stagnation reset` 이다.

23. 한계

- 이번 결과는 single split 기준이라 분산 추정이 약하다.
- hidden test도 agent당 한 번만 열었으므로, replication이나 confidence interval은 없다.
- search space를 `8` knobs로 강하게 줄였기 때문에 DOE의 장점이 일부 과소평가될 수 있다.
- 반대로 이 정도로 좁은 space에서도 `Sequential`이 쉽게 고착됐다는 점은 autoresearch loop 자체의 취약점이다.
- text는 fixed TF-IDF representation 위 실험이라, representation search까지 포함한 결론은 아니다.

24. References

구분	예시
curated landscape	awesome-autoresearch, Awesome Auto Research Tools
end-to-end systems	karpathy/autoresearch, RD-Agent, AI-Scientist
deep research	GPT Researcher
evaluation	MLE-bench, MLAgentBench, MLR-Bench
visuals	AutoML image, NAS image, Kubeflow model registry lifecycle image